Ders 26 Çalışma Özeti

Ders 26 Çalışma Özeti

Genel Konular

- Dönemin son dersi. Dönem projesi ve final sınavıyla ilgili sorular.
- Karşılaştırmaya dayalı sıralama algoritmalarının zaman ve bellek karmaşıklığı tablosu.
- Sıralama algoritmalarını karşılaştırma ölçütleri: kararlılık (stable), yerinde (in-place) sıralama, çevrim içi (online) ve çevrim dışı (offline) sıralama.
- Hangi durumda hangi algoritmanın seçileceği.
- Karşılaştırma yapmayan sıralama algoritmaları: sayma sıralaması (counting sort) ve taban sıralaması (radix sort).
- LSD ve MSD radix sort; sayılar ve dizgiler üzerinde kullanım.

Hocanın Özellikle Vurguladığı Kısımlar

- Sıralı veri birçok problemin çözümünü kolaylaştırır. Bu yüzden veri çoğu zaman önce sıralanır.
- Karşılaştırmaya dayalı sıralamada $O(n \log n)$ bir alt sınırdır. Daha iyisi tasarlanamaz.
- Karşılaştırma yapmayan algoritmalar bu sınırın altına iner. Bunun bedeli ek bellek kullanımıdır.
- Bubble sort'un en iyi durumu $O(n)$ olabilmesi için iyileştirilmiş sürümü gerekir: dış döngü, değişiklik olup olmadığını kontrol eden bir `while` döngüsü olmalıdır.
- Kararlılık her problemde gerekmez. Ama eşit değerlerin sırası önemliyse kararlı algoritma seçilmelidir.
- Counting sort, en büyük değer kadar ek dizi açar. Değerler büyükse (ör. 800, 100000) verimsizdir.
- MSD radix sort, değişken uzunluklu anahtarlarda öne çıkar.

Kısa Tekrar Notları

- Kararlı (stable) sıralama: eşit değerli elemanlar, girdideki göreli sıralarını korur.
- Yerinde (in-place) sıralama: sabit ek bellekle ($O(1)$) sıralar.
- Çevrim içi (online) sıralama: veri parça parça gelirken sıralamaya devam edebilir. Insertion sort online'dır.
- Kararlı: bubble, insertion, merge, counting, radix.
- Kararsız: selection, quick, heap.
- Yerinde: bubble, selection, insertion, quick, heap. Merge sort ve counting sort yerinde değildir.
- Counting sort: zaman $O(n + k)$, bellek $O(n + k)$. k en büyük değerdir.
- Radix sort: zaman $O(d(n + k))$. d basamak sayısı, k taban (onluk sistemde 10).

Detaylı Açıklamalar

Proje ve Sınav Soruları

- Bir öğrenci sordu: veri yapısını kurmak yavaş ama aramak hızlı olan bir çözüm seçilebilir mi?
  - Cevap: Yapı bir kez kurulup çok kez aranacaksa aramanın hızlı olması mantıklıdır.
  - Yapı sık kuruluyorsa kurulumun verimli olması gerekir.

  - Kendi varsayımınızı belirtin ve seçiminizi gerekçelendirin. Videoda da anlatın.
- İlk akla gelen kaba kuvvet (brute force) çözüm beklenmiyor. %100 en iyi çözüm de şart değil, ama iyileştirilebilecek adımlar yapılmalı.
- Projede Türkçe karakter kullanmak zorunlu değil.
- Final sınavı bir sonraki pazartesi. Bildiği kadarıyla finaller uzaktan (online) olacak.

Karşılaştırmaya Dayalı Sıralama Algoritmaları

Algoritma	En iyi	Ortalama	En kötü	Bellek (en kötü)
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quicksort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(n)$
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

- Bubble ve selection sort yer değiştirme (swapping) ile çalışır.
- Insertion sort kaydırma (shifting) ile çalışır.
- Quicksort ve heap sort yer değiştirme ile çalışır.
- Merge sort ek bellek kullanır ve veriyi kopyalar.
- Quicksort ve merge sort, böl ve yönet (divide and conquer) yaklaşımıyla Algoritma Analizi dersinde ayrıntılı işlenecek.
- Quicksort'un en kötü durumu $O(n^2)$'dir. Hoca, tersten sıralıya yakın verilerde quicksort'tan kaçınılmasını söyledi.

Karşılaştırma Ölçütleri

Kararlılık (stability): - Örnek: 3 1 5 3 2 5 4 sıralanınca 1 2 3 3 4 5 5 olur. - İlk 3 önce, ikinci 3 sonra geliyorsa algoritma kararlıdır. - Bunu garanti etmeyen algoritma kararsızdır (unstable). - $O(n \log n)$ algoritmalar içinde kararlı olan merge sort'tur.

Yerinde sıralama (in-place): - Ek bellek kullanmadan sıralayan algoritmalardır. - Gömülü sistemler gibi belleğin kısıtlı olduğu ortamlarda tercih edilir.

Online / offline: - Online algoritma, sıralama sürerken yeni gelen elemanı bozulmadan yerleştirebilir. - Insertion sort, gelen elemanı kaydırarak doğru yere koyduğu için online'dır. Diğerleri offline'dır.

Kullanılan stratejiler: - Böl ve yönet: merge sort, quicksort. - Heap yapısı: heap sort. - İç içe döngü ve artımlı (incremental) yaklaşım: selection sort, insertion sort. - Selection sort her adımda sıradaki en küçük elemanı bulur. - Insertion sort her adımda bir elemanı olması gereken yere yerleştirir.

Hangi Durumda Hangi Algoritma?

Durum	Önerilen
Küçük veri kümesi	Insertion sort (kararlı, yerinde, en iyi durumda $O(n)$)
Büyük veri kümesi	Quicksort, merge sort, heap sort
Neredeyse sıralı veri	Insertion sort
Bellek kısıtı	Heap sort, sonra quicksort
Bağlı liste	Merge sort
Paralel hesaplama	Merge sort
Kararlılık gerekli	Quicksort ve heap sort elenir

- Hoca, quicksort ve heap sort'un önbellek (cache) dostu olduğunu söyledi.
- Önbellek dostu algoritma: veriyi ana bellekten blok hâlinde L1/L2/L3 önbelleğe alıp oradan işlemekten iyi yararlanan algoritmadır.

Sayma Sıralaması (Counting Sort)

- Elemanların kaç kez geçtiğini sayar. Ek bir dizi kullanır.
- Elemanlar tekrar edebilir.
- Şart: değerler $0..k$ aralığında olmalı. $k+1$ boyutlu sayaç dizisi açılır.

Örnek dizi: `4 2 2 8 3 3 1`. En büyük değer 8.

1. En büyük değeri bul.
2. Her değerin kaç kez geçtiğini say:

Değer	0	1	2	3	4	5	6	7	8
Adet	0	1	2	2	1	0	0	0	1

3. Kümülatif toplamları al:

Değer	0	1	2	3	4	5	6	7	8
Kümülatif	0	1	3	5	6	6	6	6	7

4. Her eleman için sayacı bir azalt ve elemanı o indise yaz:
   - `4`: sayaç 6, $6-1=5$. Çıktının 5. gözüne 4 yazılır.
   - `2`: sayaç 3, $3-1=2$. 2. göze 2.
   - `2`: sayaç 2, $2-1=1$. 1. göze 2.
   - `8`: sayaç 7, $7-1=6$. 6. göze 8.
   - `3`: sayaç 5, $5-1=4$. 4. göze 3.
   - `3`: sayaç 4, $4-1=3$. 3. göze 3.
   - `1`: sayaç 1, $1-1=0$. 0. göze 1.

Sonuç: `1 2 2 3 3 4 8`.

- Sıralı dizi ayrı bir çıktı dizisinde oluşur. Yerinde değildir.
- Dört döngü var: en büyüğü bulma (n), sayma (n), kümülatif toplam (k), yerleştirme (n).
- Zaman karmaşıklığı $O(n+k)$. Bellek karmaşıklığı k'ye bağlıdır.
- Değerler $1..n^2$ aralığındaysa karmaşıklık $O(n^2)$'ye çıkar. Bu durumda avantajı kalmaz.

Taban Sıralaması (Radix Sort)

- Sayıları basamak basamak sıralar. Her basamakta counting sort mantığı kullanılır.
- Örnek: `121 432 564 23 1 45 788` (LSD, sağdan sola):

Adım	Sıralama ölçütü	Sonuç
1	Birler basamağı	121, 1, 432, 23, 564, 45, 788
2	Onlar basamağı	1, 121, 23, 432, 45, 564, 788
3	Yüzler basamağı	1, 23, 45, 121, 432, 564, 788

- Yardımcı dizinin boyutu tabana eşittir: onluk sistemde 10, ikilik sistemde 2, sekizlik sistemde 8.
- Counting sort doğrudan kullanılsaydı en büyük değer (788) kadar dizi gerekirdi.
- Karmaşıklık $O(d(n+k))$:
  - n: eleman sayısı.
  - k: taban (sayaç dizisinin boyutu).
  - d: basamak sayısı.
- d ve k sabit kabul edilirse (ör. $d=64$, $k=10$) karmaşıklık $O(n)$'ye iner. Bu, $O(n \log n)$ sınırının altıdır.
- Küçük değerli az sayıda eleman için quicksort gibi algoritmalardan iyi değildir. Basamak sayısı arttıkça avantajı belirginleşir.
- Radix sort kararlıdır. Bu yüzden önceki basamağın sıralaması bozulmaz.

Dizgilerde radix sort: - Basamak yerine karakterler kullanılır. - k: yalnız harfler için 26, ASCII için 256, Unicode için 65536. - d: en uzun dizginin uzunluğu (W). - LSD dizgi sıralaması: sabit uzunluklu dizgilerde son karakterden başlayarak her karakter için bir counting sort turu yapılır. Karmaşıklık $O(W \cdot n)$. - Kullanım alanları: büyük veritabanlarında hesap numarası, tarih, TC kimlik numarası gibi sabit uzunluklu alanlar; 128 bitlik sayılar.

LSD ve MSD Radix Sort

	LSD	MSD
Başlangıç	En düşük anlamlı basamak (sağdan)	En yüksek anlamlı basamak (soldan)
Yapı	Her basamak için bir tur	Özyinelemeli: ilk karaktere göre gruplar, her grubu ayrı sıralar
Uygun olduğu durum	Sabit uzunluklu anahtarlar	Değişken uzunluklu anahtarlar

- Değişken uzunluklu dizgilerde sağdan başlamak anlamsızdır, çünkü dizgiler hizalı değildir.
- MSD'de kısa dizgi bittiğinde artık işleme girmez. Bu da işlem sayısını azaltır.
- MSD'nin dezavantajları:
  - Belleğe rastgele eriştiği için önbellek dostu değildir.
  - Sayaç ve geçici dizi için ek bellek kullanır.
- MSD'nin avantajı: quicksort'a göre doğrusal (lineer) performans.

Ders sonrası notlar

- Altyazıda merge sort ve quicksort için de bellek $O(1)$ diye geçiyor. Tahtadaki tabloda merge sort $O(n)$, quicksort $O(n)$ yazıyor; tablo doğrudur. Quicksort özyineleme yığını için ortalamada $O(\log n)$, en kötü durumda $O(n)$ bellek kullanır.
- Hoca stratejileri sayarken counting sort'u karşılaştırmaya dayalı algoritmalar altında "hashing" stratejisi olarak andı. Counting sort karşılaştırma yapmaz ve özetleme (hashing) kullanmaz. Değerleri doğrudan dizi indisi olarak kullanır (anahtar indeksli sayma).
- Heap sort'un önbellek dostu olduğu ifadesi tartışmalıdır. Heap sort, dizide uzak indislere atladığı için önbelleği iyi kullanmaz. Pratikte quicksort önbellek açısından en iyisidir.
- Quicksort'un en kötü durumu yalnız tersten sıralı veride değil, pivot seçimine bağlıdır. İlk ya da son elemanı pivot alan sürümde sıralı veri de en kötü durumu üretir.
- Slayttaki örnekte counting sort'un yerleştirme adımı dizinin başından yapıldı. Bu şekilde eşit elemanların sırası tersine döner ve algoritma kararlı olmaz. Kararlılık için yerleştirme adımı dizinin sonundan başa doğru yapılmalıdır. Radix sort'un doğru çalışması bu kararlılığa bağlıdır.
- Counting sort'un bellek karmaşıklığı $O(n + k)$'dir: k boyutlu sayaç dizisi ve n boyutlu çıktı dizisi.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.